

Aprendizaje de Máquina

U2 Análisis de Clústeres

Héctor Maravillo

Section 1

Hierarchical Algorithms for Large Data Sets

Hierarchical Algorithms for Large Data Sets

The number of operations for the **generalized agglomerative scheme (GAS)** is of the order of N^3 , and this **cannot become less than $O(N^2)$** , even if efficient computational schemes are employed.

There are two **special hierarchical algorithms** that are most appropriate for **handling large data sets**:

- The CURE Algorithm.
- The ROCK Algorithm.

Section 2

The CURE Algorithm

The CURE Algorithm

The acronym **CURE** stands for **Clustering Using REpresentatives**.

The innovative feature of CURE is that it **represents each cluster**, C , by a set of $k > 1$ **representatives**, denoted by R_C .

By using multiple representatives for each cluster, the CURE algorithm tries to **capture the shape** of each one.

However, in order to avoid taking into account irregularities in the border of the cluster, the **initially chosen representatives** are *pushed* toward the mean of the cluster. This action is also known as **shrinking** in the sense that the volume of space defined by the representatives is shrunk toward the mean of the cluster.

The CURE Algorithm

For each C the set R_C is determined as follows:

- Select the point $\mathbf{x} \in C$ with the **maximum distance** from the mean of C and set $R_C = \{\mathbf{x}\}$.
- For $i = 2$ to $\min\{k, |C|\}$:
 - Determine $\mathbf{y} \in C - R_C$ that lies farthest from the points in R_C and set

$$R_C = R_C \cup \{\mathbf{y}\}$$

- Shrink the points $\mathbf{x} \in R_C$ toward the mean $\mathbf{m}_C$ in C by a factor α .
That is,

$$\mathbf{x} = (1 - \alpha)\mathbf{x} + \alpha\mathbf{m}_C, \quad \forall \mathbf{x} \in R_C$$

The CURE Algorithm

Given the previous definitions, the **CURE algorithm** may be viewed as a **special case** of the **GAS scheme**:

- In its original version, the representatives of a cluster, C_q , generated from the agglomeration of two clusters C_i and C_j , are determined by taking into account all the points of C_q .
- However, to **reduce the time complexity**, the representatives of C_q are selected among the $2k$ representatives of C_i and C_j

The CURE Algorithm

Such a choice is justified by the fact that the **representatives** are the **most scattered** points in each one of the clusters, C_i and C_j .

Hence, it is expected that the resulting k representative points for C_q will also be well scattered.

Once the **final number of m clusters** has been established, each point $\mathbf{x}$ in $\mathbf{X}$, which is not among the representatives of any of the final clusters, is assigned to the cluster that contains the closest to $\mathbf{x}$ representative.

The CURE Algorithm

The **worst-case time complexity** of CURE can be shown to be $O(N^2 \log_2 N)$. Clearly, this time complexity becomes prohibitive in the case of very large data sets.

The technique adopted by the CURE algorithm, in order to reduce the computational complexity, is that of **random sampling**. That is, a sample set $\mathbf{X}'$ is created from $\mathbf{X}$, by choosing randomly N' out of the N points of $\mathbf{X}$.

However, one has to ensure that the probability of missing a cluster of $\mathbf{X}$, due to this sampling, is low. This can be guaranteed if the number of points N' is sufficiently large.

The CURE Algorithm

Experiments reported in show that CURE is **sensitive** to the choice of the **parameters** k , N' , and α . Specifically,

- k must be large enough to capture the geometry of each cluster.
- N' must be higher than a certain percentage of N (for the reported experiments, N' should be at least 2.5% of N).
- The choice of the shrinking factor, a , also affects the performance of CURE.
 - For small values of a (small shrinkage), CURE exhibits a behavior similar to the MST algorithm.
 - For large values of a its performance resembles that of algorithms using a single point representative for each cluster.

The **worst-case execution time** for CURE increases quadratically with the sample size N' , that is, $O(N^2 \log_2 N')$.

Remarks

- The algorithm exhibits **low sensitivity** with respect to **outliers within the clusters**. The reason is that shrinking the scattered points toward the mean *dampens* the adverse effects due to outliers.
- For $\alpha = 1$, all representatives diminish to the mean (centroid) of the cluster.
- The CURE algorithm can reveal clusters **with nonspherical or elongated shapes**, as well as clusters of wide variance in size.

Section 3

The ROCK Algorithm

The ROCK Algorithm

The **RObust Clustering using linKs (ROCK)** algorithm is best suited for **nominal (categorical) features**.

For this type of data, it makes no sense to choose the mean (centroid) of a cluster as a representative.

In addition, the proximity between two feature vectors whose coordinates stem from a discrete data set cannot be adequately quantified by any of the l_p distances.

ROCK introduces the **idea of links**, in place of distances, to merge clusters.

Definition

Two points $\mathbf{x}, \mathbf{y} \in \mathbf{X}$ are considered **neighbors** if

$$s(\mathbf{x}, \mathbf{y}) \geq \theta$$

where $s(\cdot)$ is an appropriately chosen **similarity function** and θ is a **user-defined parameter**, which defines the similarity level according to which two points can be considered **similar**.

Let $link(\mathbf{x}, \mathbf{y})$ be the **number of common neighbors** between $\mathbf{x}$ and $\mathbf{y}$.

Consider the graph whose vertices are the points in $\mathbf{X}$ and whose edges connect points that are neighbors.

Then, it can easily be determined that $link(\mathbf{x}, \mathbf{y})$ may be viewed as the **number of distinct paths of length 2** connecting $\mathbf{x}$ and $\mathbf{y}$.

The ROCK Algorithm

Assume now that there exists a function $f(\theta) < 1$ that is dependent on the data set as well as on the type of clusters we are interested in, with the following property:

- Each point assigned to a cluster C_i has approximately $|C_i|^{f(\theta)}$ neighbors in C_i .

The ROCK Algorithm

Assuming that cluster C_i is large enough and that points outside C_i result in a very small number of links to the points of C_i , each point in C_i contributes approximately to $|C_i|^{2f(\theta)}$ links (possible links to a point).

Thus, the **expected total number of links** among all pairs in C_i is the potential number of connections that can exist with a single point depends on the number of points in the cluster; in other words,

$$|C_i| |C_i|^{1+2f(\theta)} = |C_i|^{1+2f(\theta)}$$

The ROCK Algorithm

The **closeness** between two clusters is assessed by the function

$$g(C_i, C_j) = \frac{\text{link}(C_i, C_j)}{|C_i + C_j|^{1+2f(\theta)} - |C_i|^{1+2f(\theta)} - |C_j|^{1+2f(\theta)}}$$

where

$$\text{link}(C_i, C_j) = \sum_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} \text{link}(\mathbf{x}, \mathbf{y})$$

Note that the denominator in this fraction is the **expected total number of links** between the two clusters.

The ROCK Algorithm

Clearly, the larger the value of $g(\cdot)$ the more similar the clusters C_i and C_j .

The number of clusters, m , is a **user-defined parameter**.

At each iteration, the clusters with **maximum $g(\cdot)$ are merged**.

The **procedure stops** when the number of clusters formed becomes equal to the desired number of clusters, m , or the number of links between every pair of clusters in a clustering R_t is 0.

The ROCK Algorithm

- Some authors propose use

$$f(\theta) = \frac{(1 - \theta)}{1 + \theta}, \quad \theta < 1$$

- For large values of N , the **worst-case time complexity** of ROCK is similar to that of CURE.
- Under certain conditions, the clusters produced by the ROCK algorithm are the **connected components** of a certain graph, called **link graph**. Based on the above result, a quick version of ROCK, called **QROCK**, is described, which simply **identifies the connected components** of the link graph.

Section 4

References

References

- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009